

Project Manager

Your private deep-dive — every feature explained with WHY, not just WHAT.

RBAC + Roles

Repository Pattern

Service Layer

Approval Workflow

OTP-less Auth

Laravel Pint

CHAPTER 01

What Is Project Manager & How Is It Different from FruGo?

Project Manager is the **back-office admin tool**. Normal customers never see it. It is used by:

- **Admins** — can approve/reject products, manage all users, see everything
- **Product Managers** — can add and edit their own products (which then wait for admin approval)

FruGo is the **customer shop**. FruGo only shows products that were approved in Project Manager. The two apps share the same database — if an admin approves a product in Project Manager, it immediately appears in FruGo.

ANALOGY

Think of a supermarket. **Project Manager** = the warehouse and HR office (staff-only, admin tasks). **FruGo** = the actual shop floor (customers, carts, checkout). The warehouse decides which products are ready to sell; the shop floor is where they actually get sold.

CHAPTER 02

Multi-Role RBAC — Who Can Do What?

RBAC stands for **Role-Based Access Control**. Instead of checking "is this user #42?", we check "what role does this user have?"

CONCEPT

There are 3 roles: **admin**, **product_manager**, **customer**. Each role unlocks different parts of the app. The role is stored as a plain string in the `users.role` column.

How it works in code — three layers:

app/Models/User.php — role constants

PHP

```
// Constants prevent typo bugs: 'Admi' or 'ADMIN' would silently fail
const ROLE_ADMIN          = 'admin';
const ROLE_PRODUCT_MANAGER = 'product_manager';
const ROLE_CUSTOMER       = 'customer';

// Helper method — callable anywhere: $user->isAdmin()
public function isAdmin(): bool
{
    return $this->role === self::ROLE_ADMIN;
}
```

app/Http/Middleware/CheckRole.php

PHP

```
public function handle(Request $request, Closure $next, string ...$roles): Response
{
    // auth() gives the logged-in user. hasAnyRole() checks if their role
    // is in the allowed $roles list. If not: 403 Forbidden.
    if (!auth()->check() || !auth()->user()->hasAnyRole($roles)) {
        abort(403, 'Unauthorized');
    }
    return $next($request);
}
```

routes/web.php — applying middleware to routes

PHP

```
// Only admins can reach /admin routes
Route::middleware(['auth', 'role:admin'])->prefix('admin')->group(function() {
```

```
Route::get('/users', [AdminController::class, 'index']);

});

// Admins AND product_managers can reach /products
Route::middleware(['auth', 'role:admin,product_manager'])->group(function() {
    Route::resource('products', ProductController::class);
});
```

WHY THIS WAY?

We register the middleware alias `role` in `bootstrap/app.php` so the route file can write `role:admin` (short) instead of the full class name. If we didn't use middleware, we'd have to write `if (!$user->isAdmin()) abort(403)` at the top of *every single controller method* — messy and easy to forget.

ROLE	CAN DO	CANNOT DO
admin	Approve/reject products, manage all users, see all products	Nothing blocked
product_manager	Create/edit/delete own products, view own dashboard	Cannot approve products, cannot see other managers' products, cannot manage users
customer	Browse FruGo shop, add to cart, checkout, view orders	Cannot access /products or /admin at all

CHAPTER 03

Password Auth (Project Manager) vs OTP Auth (FruGo)

Project Manager uses **classic email + password**. FruGo uses **OTP (One-Time Password)** with no password at all. Same codebase, same users table, different auth strategy for different user types.

Project Manager — Password Login

- Staff accounts pre-created by admin
- Email + password submitted on login form
- Laravel's built-in `Auth::attempt()`

FruGo — OTP Login

- Customer just enters their email
- 6-digit code sent to that email
- Code expires in 10 minutes

- Password hashed with bcrypt (12 rounds)
- Rate-limited: 5 attempts before lockout
- Session regenerated on login to prevent fixation

- Token deleted immediately after use
- Account auto-created on first login
- Zero friction — no password to forget

WHY TWO DIFFERENT AUTH SYSTEMS?

Staff (admins, product managers) are trusted, pre-registered people — password auth is fine, they won't forget it. Customers are anonymous members of the public — asking them to create and remember a password just to buy fruits is friction that kills conversions. OTP removes the barrier entirely. Same app, two different user experiences.

CHAPTER 04

Repository Pattern — Separating Database Queries from Business Logic

Without a Repository, controllers get messy: they handle HTTP input, write SQL queries, AND apply business rules — all mixed together. A repository is a dedicated class that owns all database queries.

ANALOGY

Imagine a library. You don't climb the shelves yourself. You talk to a **librarian** (the Repository) and say "I want books by author X, from 1990-2000, sorted by title." The librarian knows exactly where everything is. You just say what you want. The Repository is the librarian — it knows the database layout so the controller doesn't have to.

app/Repositories/Contracts/ProductRepositoryInterface.php

PHP

```
interface ProductRepositoryInterface
{
    // Defines what operations a product repository MUST support
    // The controller only knows this interface – not the implementation
    public function paginate(int $perPage = 10): LengthAwarePaginator;
    public function find(int $id): ?Product;
    public function create(array $data): Product;
```

```

public function update(Product $product, array $data): bool;
public function delete(Product $product): bool;
public function updateStatus(Product $product, string $status): bool;
}

```

app/Repositories/ProductRepository.php – the actual queries

PHP

```

public function paginate(int $perPage = 10): LengthAwarePaginator
{
    // with('creator') = eager-load the user who created the product
    // Avoids N+1: without this, showing 10 products = 11 queries
    return Product::with('creator:id,name')
        ->latest()
        ->paginate($perPage);
}

```

CONCEPT – N+1 QUERY PROBLEM

If you show 10 products without eager loading, Eloquent runs 1 query to get the products, then 1 more query for each product to find its creator — that's 11 queries. With `with('creator')`, it's just 2 queries total: one for products, one for all their creators combined. Always eager-load related data you're going to display.

app/Providers/RepositoryServiceProvider.php – binding the interface

PHP

```

// This tells Laravel: "whenever someone type-hints ProductRepositoryInterface,
// give them a ProductRepository instance"
$this->app->bind(
    ProductRepositoryInterface::class,
    ProductRepository::class
);

```

WHY THE INTERFACE?

The interface is a **contract**. The controller imports `ProductRepositoryInterface`, not `ProductRepository`. This means you could swap the repository for a different one (e.g.

`RedisProductRepository`) without touching the controller. In tests, you can inject a fake repository that returns dummy data without hitting the database.

CHAPTER 05

Service Layer — Where Business Rules Live

The controller handles HTTP (input + output). The repository handles database queries. The **service** handles business logic — the rules about *what is allowed*.

app/Services/ProductService.php — approval + ownership

PHP

```
public function approve(Product $product): bool
{
    // Business rule: only pending products can be approved
    // The controller doesn't know this rule — the service enforces it
    if ($product->status !== 'pending') {
        throw new InvalidStatusTransitionException();
    }
    return $this->repository->updateStatus($product, 'approved');
}

public function canModify(User $user, Product $product): bool
{
    // Business rule: admins can modify anything; PMs only their own products
    return $user->isAdmin() || $product->created_by === $user->id;
}
```

WHY NOT JUST PUT THIS IN THE CONTROLLER?

If you put these rules in the controller, every other place that approves a product would have to copy those rules. By putting them in the service, there is **one place** that decides what "approve" means. Tests for business logic are simpler too — no HTTP requests needed, just call the service directly.

Without Service (bad)

With Service (good)

Controller checks status AND calls repository AND sends email AND logs. If you need to approve from a command line script too, you copy all that logic there. Two copies = two bugs.

Controller calls `$service->approve($product)`. The service does the checks. The CLI script also calls `$service->approve()`. One place, one truth.

CHAPTER 06

Product Approval Workflow

Every product goes through a state machine: **pending** → **approved** or **pending** → **rejected**. Only admins can move the state. This is stored in the `products.status` column.

1 PM creates product

ProductController@store calls ProductService@create. Status automatically set to 'pending'.

2 Admin sees pending products

Admin dashboard filters where status = 'pending'. PM cannot see this list.

3 Admin clicks Approve or Reject

ProductController@approve calls ProductService@approve which calls ProductRepository@updateStatus.

4 Status changes in database

UPDATE products SET status = 'approved' WHERE id = ?

5 FruGo shop shows it

ShopController only queries approved products — Product::approved()->paginate(12)

app/Models/Product.php — the 'approved' scope

PHP

```
// Eloquent 'scopes' are reusable query conditions
// scopeApproved lets you write Product::approved() instead of
// Product::where('status', 'approved') every single time
public function scopeApproved(BUILDER $query): void
{
    $query->where('status', 'approved');
}

public function scopePending(BUILDER $query): void
{

```

```
$query->where('status', 'pending');  
}
```

EXAMPLE – WHY SOFT DELETES MATTER

Product has `SoftDeletes` trait. When a PM "deletes" a product, it sets `deleted_at` to now. The product disappears from all normal queries, but the record stays in the database. If there's a dispute ("I never deleted that!") or orders reference deleted products, the data is still there. Hard delete = gone forever.

CHAPTER 07

Admin User Management

Only admins can create new users and assign roles. There is no public registration for Project Manager — accounts are invitation-only.

AdminController@store – creating a user with a role

PHP

```
public function store(UserRequest $request): RedirectResponse  
{  
    $user = User::create([  
        'name'      => $request->name,  
        'email'     => $request->email,  
        'password' => Hash::make($request->password),  
        // Role comes from a <select> dropdown – only admin,product_manager allowed  
        'role'      => $request->role,  
    ]);  
  
    return redirect()->route('admin.users.index')  
        ->with('success', 'User created');  
}
```

WARNING – NEVER TRUST ROLE FROM FORM DIRECTLY

The role dropdown only shows `admin` and `product_manager` as options. But a hacker could manually send `role=superuser` in a POST request. That's why `UserRequest` validates `rule('role',`

`['required', 'in:admin,product_manager']])` — it rejects any value not in that list before the controller even sees it.

CHAPTER 08

Form Requests — Where Validation Lives

Laravel's Form Requests are dedicated classes that validate incoming HTTP data before it reaches your controller. Instead of writing `$request->validate([...])` inside the controller, you move it to a separate class.

app/Http/Requests/ProductRequest.php

PHP

```
class ProductRequest extends FormRequest
{
    public function authorize(): bool
    {
        // Only logged-in users with the right role can submit this form
        return auth()->check() && auth()->user()->hasAnyRole([...]);
    }

    public function rules(): array
    {
        return [
            'title'      => ['required', 'string', 'max:255'],
            'description' => ['required', 'string'],
            'price'      => ['required', 'numeric', 'min:0'],
            // nullable: image is optional when editing an existing product
            'image'      => ['nullable', 'image', 'max:2048'],
        ];
    }

    protected function prepareForValidation(): void
    {
        // Runs BEFORE rules() - sanitise the description HTML here
        $this->merge([
            'description' => strip_tags($this->description, ['b', 'i', 'u', 'p', 'br']),
        ]);
    }
}
```

```
    });  
}  
}
```

WHY A SEPARATE CLASS INSTEAD OF INLINE VALIDATE()?

When you type-hint `ProductRequest $request` in the controller method, Laravel automatically validates *before the method body runs*. If validation fails, Laravel redirects back with errors automatically — zero extra code. The controller method only runs when the data is already valid. Cleaner controller, reusable validation.

CHAPTER 09

Rich-Text XSS Protection

Products have a description field that uses the Quill rich-text editor. Quill produces HTML like `<b>Fresh</b>`. But a malicious user could submit `<script>steal(document.cookie)</script>` instead of real content. If that gets stored and displayed, it runs in every visitor's browser — that's an XSS (Cross-Site Scripting) attack.

CONCEPT – XSS

XSS = an attacker injects JavaScript into your page. When other users load the page, the script runs in *their* browser with access to their cookies, session tokens, and passwords. The fix: never render user-supplied HTML without sanitising it first.

Two-layer defence

PHP + Blade

```
// Layer 1: strip_tags() in ProductRequest::prepareForValidation()  
// Allows only safe formatting tags, kills everything else  
strip_tags($html, '<b><i><u><p><br><ul><li><strong><em>')  
  
// Layer 2: in Blade templates  
// {{ $product->title }} – SAFE: Blade HTML-encodes the output  
// {!! $product->description !!} – UNSAFE if not sanitised first!  
// We use {!! !!} ONLY for description because we already strip_tag'd it
```

```
// What Blade does: {{ $title }} becomes htmlspecialchars($title)
// "<script>" becomes "&lt;script&gt;" – shown as text, not run as code
```

EXAMPLE

User submits: `<b>Fresh</b><script>alert('hacked')</script>`

After `strip_tags` : `<b>Fresh</b>`

The script tag is gone. Only the safe formatting tag remains. The product description now shows "Fresh" in bold, safely.

CHAPTER 10

Full-Text Search

The product index supports searching by keyword. There are two ways to do this in MySQL — a basic LIKE query and a proper FULLTEXT index.

LIKE search (simple, slower)

```
WHERE title LIKE '%mango%'
```

- The % wildcard means "anything before/after"
- Scans every row in the table
- No index can help — always a full table scan
- Fine for small tables (<1000 rows)

FULLTEXT index (better)

```
MATCH(title,description) AGAINST('mango')
```

- MySQL builds a word-level index
- Searches thousands of rows instantly
- Returns relevance score (best matches first)
- Handles multiple words automatically

database/migrations — adding FULLTEXT index

PHP

```
// In the products migration
$table->fullText(['title', 'description']);

// In ProductRepository
public function search(string $query): Builder
{
```

```
return Product::whereFullText(['title', 'description'], $query);  
}
```

CHAPTER 11

PSR-12 Code Standards & Laravel Pint

PSR-12 is a set of formatting rules agreed upon by the PHP community. Things like: opening braces go on the same line as the class, 4 spaces indent (not tabs), one blank line between methods. Laravel Pint is a tool that automatically fixes your code to follow these rules.

CONCEPT

PSR stands for "PHP Standards Recommendation". PSR-4 = autoloading (where class files live). PSR-12 = code formatting style. When everyone on a team uses the same style, reading other people's code looks like reading your own. No arguments about tabs vs spaces ever again.

Running the linter

BASH

```
# Fix all style issues automatically  
./vendor/bin/pint  
  
# Or via Makefile shortcut  
make lint  
  
# Check without fixing (shows what would change)  
./vendor/bin/pint --test
```

WHY ENFORCE CODE STYLE?

When you open a file written by someone else, you shouldn't have to puzzle out their formatting choices. Consistent style means your eyes know exactly where to look. Also, git diffs become meaningful — they show real changes, not reformatting noise. `declare(strict_types=1)` at the top

of each file makes PHP error on type mismatches (e.g. passing a string where an integer is expected) rather than silently converting it.

CHAPTER 12

Docker — 4 Services, One Command

Without Docker, every developer needs to install PHP, Nginx, MySQL, and MailHog on their own machine — and deal with different versions and configs. Docker packages each service in a container so everyone gets the exact same environment.

SERVICE	WHAT IT DOES	PORT
<code>pm_nginx</code>	Web server. Receives HTTP requests, serves static files (CSS/JS), forwards PHP requests to <code>pm_app</code>	8080
<code>pm_app</code>	PHP-FPM. Runs your Laravel app code. Nginx talks to this via FastCGI on port 9000	9000 (internal)
<code>pm_db</code>	MySQL 8 database. Stores all tables (users, products, orders...)	3306 (internal)
<code>pm_mailhog</code>	Fake email server. Catches all emails your app sends — viewable at localhost:8025	8025 (web UI) / 1025 (SMTP)

ANALOGY

Docker Compose is like a restaurant kitchen with 4 stations. Nginx is the front-of-house (takes orders). PHP-FPM is the chef (cooks the food). MySQL is the pantry (stores ingredients). MailHog is the test kitchen (tastes things before serving). `docker compose up` opens the whole restaurant at once.

Common Docker commands

BASH

```
# Start all 4 services
make up # or: docker compose up -d

# See PHP/Nginx logs in real time
```

```
docker logs pm_app -f

# Run artisan commands inside the PHP container
docker exec pm_app php artisan migrate

# Open a shell inside the container
make shell # or: docker exec -it pm_app sh

# Fix storage permissions (do this if you see permission denied errors)
docker exec pm_app chown -R www-data:www-data /var/www/storage
```

CHAPTER 13

MailHog — Catch All Emails Locally

Project Manager sends emails (password reset, notifications). In development, you don't want real emails going to real inboxes. MailHog is a local "email trap" — your app thinks it's sending emails to a real SMTP server, but MailHog silently catches them all.

EXAMPLE — HOW TO VIEW AN EMAIL

- Start Docker: `make up`
- Trigger an email (e.g. request password reset)
- Open `http://localhost:8025` in your browser
- You'll see the email in the inbox — click to open, see subject/body/HTML

.env — MailHog configuration

ENV

```
# Laravel talks to MailHog on port 1025 (SMTP)
MAIL_MAILER=smtp
MAIL_HOST=mailhog      # 'mailhog' = Docker service name, resolves internally
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_FROM_ADDRESS="noreply@projectmanager.test"
```

WHY MAILHOG AS THE HOSTNAME?

Inside Docker Compose, containers can reach each other by **service name**. The mail service is named `mailhog` in `docker-compose.yml`, so your PHP container can connect to it at `mailhog:1025`. On your laptop, you'd use `localhost:1025`, but inside Docker the service name is used instead of an IP address — Docker handles the routing automatically.

Project Manager — Personal Learning Guide

Laravel 11 • PHP 8.2 • MySQL 8 • Docker • RBAC • Repository Pattern